Seminar Pitch Script — Agentic AI: Autonomous Goal-Oriented Intelligent Systems for Planning, Reasoning and Task Execution

Student: Korde Ayush Sandeep | PRN 72406771M | T.E. IT, Sem I, AY 2026–27 Guide: Prof. V. A. Khairnar

A full spoken script tying together the PPT (22 slides), the synopsis, and the live prototype demo into one delivery. Target total time **10–15 minutes**, including the 3–5 minute live demo. Cues in `[brackets]` mark which PPT slide(s) you should be on; *italics* are stage directions, not spoken words.

---

0. Cold open (30 sec) — before Slide 1 fully settles

> "Good [morning/afternoon]. Everyone here has used a chatbot. You type a question, it answers, the conversation ends. What I'm presenting today is *not* that. I'm going to show you a system that takes a goal, breaks it into steps, checks its own work, asks for your permission before doing anything risky — and I'm not just going to describe this architecture, I'm going to run it live and let you watch it think."

Pause. Advance to Slide 1.

---

1. Title (Slide 1) — 15 sec

> "My topic is Agentic AI: Autonomous Goal-Oriented Intelligent Systems for Planning, Reasoning and Task Execution. Guided by Prof. V. A. Khairnar."

---

2. Outline (Slide 2) — 20 sec

> "Here's the path: what Agentic AI actually is, why it matters, what the literature is missing, the architecture I'm proposing to fill that gap, and then — the part I'm most excited about — a live demonstration of a working prototype built on exactly this architecture."

---

3. Introduction (Slides 3–6) — 1.5 min

[Slide 3 — Introduction]

> "AI systems have gone through three stages. First, rule-based automation — fixed logic, no adaptation. Second, generative AI and large language models — they produce fluent text or code in response to a prompt, but they don't act in the world across multiple steps by themselves. Third — Agentic AI. Here, an LLM acts as a *controller* inside a loop: it plans, retrieves knowledge, picks tools, executes actions, observes what happened, and decides whether to continue, retry, or stop.
>
> The distinction that matters: a chatbot can *explain* how to book a venue. An agentic system can actually *attempt* to book it. The moment a system can act instead of just describe, reliability and safety stop being optional — they become the core engineering problem."

[Slide 4 — Motivation]

> "Three things motivated this seminar. LLMs are increasingly wired into real tools and APIs, so one unverified action — a wrong email, a bad calculation — has real consequences; risk needs to be checked *before* execution, not after. Second, the literature covers planning, reasoning-acting loops, memory and retrieval, and multi-agent systems individually — but rarely packages them into one teachable loop a student can actually build. Third, this is exactly the tooling T.E. IT graduates will meet in industry — understanding the control loop, not just prompting a model, is the transferable skill."

[Slide 5 — Objective]

> "Five objectives: define Agentic AI precisely against conventional and generative AI; survey the core components of an agentic loop; identify a concrete research gap; propose a reliability-oriented architecture; and demonstrate it end-to-end in a small, safe prototype."

[Slide 6 — Problem Statement]

> "Existing research treats planning, retrieval, tool use, multi-agent collaboration, and security risk as separate concerns, studied in isolation. A practical gap remains in integrating these into a minimal, teachable architecture that combines pre-action risk assessment, bounded human oversight, and post-action verification in one demonstrable loop. That gap is what this seminar addresses."

---

4. Related Work & Research Gap (Slides 7–9) — 1.5 min

[Slides 7–8 — Related Work / Literature Survey]

> "Seven papers ground this work. Acharya et al.'s 2025 IEEE Access survey frames what Agentic AI *is*. Dong et al. and Wang et al. frame the LLM as a controller over perception, memory, planning, and action. Yao et al.'s ReAct — reasoning plus acting — is the mechanism underneath my planning stage. Li et al. survey multi-agent workflows, which I deliberately scope *out* of this seminar to get a

single agent's reliability right first. Gan et al.'s 2026 ACM paper catalogs the security and privacy threats that show up once agents act autonomously — that's what motivates my risk gate. And Lewis et al.'s RAG paper is the retrieval mechanism grounding every action in real context."

[Slide 9 — Research Gap]

> "Put together: these seven papers cover *capability* — can the agent plan, retrieve, act — very well. What's missing is *reliability governance* — should this specific action be allowed right now, and was it actually verified afterward — integrated into the same loop. That integration gap is what I'm targeting, not a new individual capability."

---

5. Proposed Architecture & Working Flow (Slides 10–13) — 2.5 min

[Slide 10 — Proposed Methodology]

> "My approach has five parts: understand the goal, decompose and plan using a ReAct-style reasoning-acting loop, ground every step in retrieved knowledge before acting, gate every action through a pre-action risk check with mandatory human approval for anything high-risk, and verify every result against an expected outcome afterward, with a bounded retry before escalating to a human."

[Slide 11 — Proposed Agentic AI Architecture] *(point at the diagram)*

> "Here's the architecture as five layers. Input — the user's goal. Planning and Reasoning — goal analysis, the LLM, task decomposition, the planner. Knowledge and Tools — memory plus retrieval, and the tool manager. Safety and Oversight — the risk gate and human approval, sitting *before* execution. And Execution and Verification — the action runs, gets observed, gets verified, and a bounded retry loop feeds failures back to the planner.
>
> Two design choices I want you to notice. Planning is visually

> separated from Execution — everything above the safety layer decides *what to do*; everything below *does it*. And human approval sits *before* execution, never after. A risky action never gets a chance to run and get apologized for later."

[Slide 12 — Working Flow]

> "This is the same logic as a step-by-step flowchart, including the explicit decision points: is the risk high? If so, wait for approval — denied means terminate safely, approved means proceed. After execution: did verification pass? If not, and retries remain, replan and try again. If retries are exhausted, escalate to a human instead of looping forever."

[Slide 13 — Agentic AI Algorithm]

> "And here's that same loop as an algorithm: plan, retrieve, select a tool, check risk, get human approval if needed, execute, observe, verify, and a bounded retry — `MAX_RETRIES` — before escalating or returning the result. This is provably terminating by construction. There is no path to an infinite loop."

---

6. Live Prototype Demo (3–5 min) — the centerpiece

Switch to the browser. Have `streamlit run app.py` already running.

> "Everything I just described on slides isn't just a diagram — I built it. This is a working Python prototype implementing exactly this loop. Let me show you, live, with one click."

Click ▶ RUN FULL DEMO.

> "This one button runs all four scenarios that prove the architecture, back to back, deterministically — no scripted video, this is running right now."

Wait for the Demo Summary: `4 / 4 scenarios passed`.

> "Four green checks. Let me open each one and show you what actually happened underneath."

Expand Scenario 1 — Normal Execution.

> "Goal: prepare an event brief, calculate capacity, build a checklist. Watch the trace: it decomposed into three subtasks, retrieved real context from a local knowledge base for each one — that's the RAG step — selected the right tool, checked the risk as LOW, executed, observed the result, and verified it numerically against the expected answer before marking it complete. This is agency, not a chat reply."

Expand Scenario 2 — High-Risk Approval.

> "Goal: send the event announcement to all participants. Watch — it stops. Risk is classified HIGH, and it will not proceed without approval. In a live manual session I'd click Approve myself; here it's simulated for the demo, logged explicitly as simulated so it's never confused with a real decision — then it executes a *mock* send. Nothing was ever really sent — I can show you the mock log at the bottom of the page proving that."

Expand Scenario 3 — High-Risk Denial.

> "Same goal, but this time the decision is Deny. Look at the trace — there is no execute step at all for that tool. The action is refused before it ever runs. This is the one non-negotiable rule of the whole system: a denied high-risk action never executes, full stop."

Expand Scenario 4 — Bounded Retry → Escalation.

> "Same brief goal, but I've told the verifier to force the capacity calculation to fail. Watch: it fails, replans, retries — fails, replans, retries again — fails a third time, and stops. Exactly two retries, three total attempts, then it escalates to a human instead of spinning forever. `MAX_RETRIES` is enforced structurally, not by luck."

Return to slides / stand back up.

> "That's the seminar's proposed architecture, running, not simulated in a video, not hard-coded to succeed — actually executing the loop, live, in front of you."

---

7. Implementation, HW/SW, Evaluation, Applications, Challenges (Slides 14–18) — 1.5 min

[Slide 14 — Implementation Details]

> "The prototype uses a small local knowledge base, four allow-listed tools — a calculator, a checklist generator, a knowledge search tool, and one deliberately high-risk mock notification tool — and a provider-agnostic LLM layer, so it can run against a local model, an API, or, as it's configured right now, a deterministic fallback that makes the demo reproducible every single time."

[Slide 15 — Hardware / Software]

> "Nothing exotic: Python, a laptop, no GPU required, no database, no external services contacted."

[Slide 16 — Evaluation Framework]

> "I want to be precise here: I have not run a controlled experiment, so I'm not going to show you invented accuracy percentages. What I have is a proposed evaluation framework — task success rate, planning accuracy, tool-call success rate, retrieval grounding, verification accuracy, human intervention rate, and critically, safety violation rate, which my own tests already confirm is zero across every scenario in this prototype."

[Slide 17 — Applications]

> "Coding assistants that verify before presenting a diff. Enterprise copilots that gate customer-facing actions behind approval. Research assistants that flag claims needing human check. Personal task

agents where payments and messages are never sent without explicit consent."

[Slide 18 — Challenges and Limitations]

> "Risk classification is context-dependent, not solved by a fixed rule forever. Verification is only as good as how clearly you specify the expected outcome. And I'll say this plainly: my own evaluation here is structural, not a controlled experiment — that's a stated limitation, not something I'm hiding."

---

8. Conclusion (Slide 19) — 45 sec

> "Agentic AI extends LLMs from single-turn generation to autonomous, multi-step, goal-directed behavior. This seminar identified a real gap — the missing integration of capability with reliability governance — and proposed, then actually built, an architecture that separates planning from execution and pre-action safety from post-action verification, with bounded human oversight throughout. Future work: extend to multi-agent settings while keeping the same guarantees, and run a real empirical evaluation once there's a production-scale deployment to measure."

---

9. References & Thank You (Slides 20–22) — 20 sec

> "Seven references, all independently verified — including one correction I made after cross-checking a citation directly against the publisher's own metadata rather than trusting a draft title. Thank you. I'm happy to take questions."

---

10. Anticipated questions — have these ready

"What's your contribution beyond just using an LLM?"

> "The contribution is the control layer *around* the LLM — structured planning, tool selection, pre-action risk gating, mandatory human approval for high-risk actions, post-action verification, and bounded recovery. The LLM only ever *proposes* a structured action; it never executes anything directly. The tool registry, the risk gate, and the verifier are what actually decide what happens, and the LLM cannot talk its way past the gate — risk is read from the tool's own registration, never from whatever the model claims about its own action."

"Why didn't you use LangChain / a big agent framework?"

> "For a seminar prototype, the goal is to *explain my own control loop*, not to explain someone else's framework. A hand-rolled controller is small enough that every line is defensible in a viva, and it keeps the intellectual contribution — the reliability loop — visible instead of buried inside a third-party library."

"Is this production-ready?"

> "No, and I'm not claiming that. It's a scoped academic prototype: no database, no auth, no real external actions, in-memory state. Every one of those is a deliberate non-goal, documented up front, not a shortcut I'm hiding."

"Why does the demo use a deterministic provider instead of a real LLM?"

> "Two reasons. First, practically: this machine has neither a local Ollama server nor an API key configured, and I didn't want the seminar demo depending on live internet access. Second, architecturally: a real LLM can legitimately phrase or order its output differently between runs, which would undermine a live demo that needs to reliably show the same four outcomes every time. The

> interface is provider-agnostic — swapping in Ollama or an API key needs zero code changes — I just chose the reproducible option for a live audience."

"How do you know the retry logic doesn't loop forever?"

> "It's bounded by construction, not by luck — `MAX_RETRIES` is checked before every retry, and I have an automated test that specifically proves exactly `MAX_RETRIES + 1` attempts occur and no more, which you just watched happen live in Scenario 4."

"What was the real research gap, precisely?"

> "Existing surveys cover planning, retrieval, tool use, and multi-agent coordination as capabilities, and a separate line of work catalogs the security risks of letting agents act. But integrating pre-action risk assessment, bounded human oversight, and post-action verification into *one* minimal, teachable, demonstrable loop — that specific integration — is what's under-addressed, and what this seminar targets."

Timing summary

Section	Time
Cold open + Title + Outline	~1 min
Introduction (4 slides)	1.5 min
Related Work + Research Gap	1.5 min
Architecture + Working Flow + Algorithm	2.5 min
Live prototype demo	**3–5 min**
Implementation → Challenges	1.5 min
Conclusion + References + Thank You	~1 min

Total	**~12–14 min**, within the 10–15 min slot

Q&A time is separate and not counted above — rehearse the six anticipated answers so they come out naturally rather than read.